Handling Concurrent Booking Requests

How Ambula prevents double-booking when two patients request the same slot simultaneously.

Doctor appointment slots are a **finite, shared resource**. Without protection, two patients requesting the same slot simultaneously could both succeed — causing a double-booking. Ambula prevents this with a layered strategy.

Layer 1 — MongoDB ACID Transactions (Primary Guard)

Every booking flows through a MongoDB multi-document transaction on a replica-set session (bookingService.js). The slot status check and the status update happen atomically inside the same transaction:

```
const session = await mongoose.startSession();
session.startTransaction();

// 1. Read slot inside transaction — acquires document lock
const slot = await Slot.findById(slotId).session(session);
if (slot.status !== 'available')
throw new Error('Slot no longer available'); // fails fast

// 2. Create Appointment document atomically
const [appt] = await Appointment.create([{ ... }], { session });

// 3. Mark slot 'booked' — same transaction, same commit
await Slot.findByIdAndUpdate(slotId,
{ status: 'booked', appointmentId: appt._id },
{ session });

await session.commitTransaction();
// Concurrent request hits 'booked' status → abortTransaction()
```

Why this works: MongoDB's WiredTiger engine serialises writes to the same document. Only ONE transaction can commit *available* → *booked*. The second concurrent request reads the updated status inside its own transaction and throws, triggering **abortTransaction()**.

Layer 2 — Real-time UI Updates via Socket.IO

After a successful commit, the server emits a **slot-booked** event to the doctor's Socket.IO room. All connected patients viewing that doctor's calendar receive the update instantly and the slot is grayed out — reducing the window where a second user even attempts to book.

```
// Fired AFTER commitTransaction() — best-effort, non-blocking
appointmentNS
.to(`doctor-${doctorId}`)
.emit('slot-booked', { slotId, doctorId });
```

Layer 3 — Slot Status Enum Guard

The Slot schema enforces *status* ∈ *{available, booked, locked, blocked}*. A slot can only transition from *available* to *booked* once. A compound index on **(doctorId, date, status)** makes availability queries fast and consistent under load.

Layer 4 — Waitlist for Cancelled Slots

When an appointment is cancelled, the slot is atomically returned to *available* in the same transaction. The first *waiting* entry in the Waitlist collection is promoted to *notified* and a Notification document is created — all in the same commit. The patient receives an in-app alert to re-book quickly.

Flow Summary

Step	Action	Outcome on Conflict
1	Transaction opens, slot read	Document lock acquired
2	Status checked: available?	If 'booked' → throw → abort
3	Appointment created atomically	Only inside session
4	Slot status → 'booked' committed	Concurrent tx sees 'booked'
5	Socket.IO slot-booked emitted	UI updates for all viewers